

Document number: P0117R0

Date: 2015-09-25

Project: Programming Language C++, Library Evolution Working Group

Reply-to: Robert Kawulak <Robert.Kawulak@gmail.com>

Generic to_string/to_wstring functions

I. Table of Contents

- II. [Introduction](#)
- III. [Motivation](#)
- IV. [Impact on the Standard](#)
- V. [Design Decisions](#)
- VI. [Technical Specifications](#)

II. Introduction

The paper proposes extending the Standard Library with function templates equivalent to the following:

```
template<class Ch, class Tr = char_traits<Ch>, class Alloc = allocator<Ch>, class... Args>
basic_string<Ch,Tr,Alloc> to_basic_string(Args&&... args)
{
    basic_ostringstream<Ch,Tr,Alloc> stream;
    stream.exceptions(ios_base::failbit | ios_base::badbit);
    stream << ... << forward<Args>(args);
    return stream.str();
}

template<class... Args>
string to_string(Args&&... args)
{
    return to_basic_string<char>(forward<Args>(args)...);
}

template<class... Args>
wstring to_wstring(Args&&... args)
{
    return to_basic_string<wchar_t>(forward<Args>(args)...);
}
```

III. Motivation

For a long time C++ programmers have been looking for an easy way to convert an object into its string representation. A typical answer to this problem was to create a local `ostringstream`, insert the object into the stream, and then obtain the resulting string with the `str` member function. This solution is simple, safe, flexible and extensible, though definitely too verbose for something that rather ought to be a single function call. C++11 provided (a partial) solution in the form of a set of overloaded `to_string/to_wstring` functions. Unfortunately, these are limited only to the built-in numeric types. Non-standard solutions exist too – most notably `boost::lexical_cast`, which offers two-way conversion of objects and strings, but lacks any formatting control.

This paper proposes a solution that:

- generalizes the existing `to_string/to_wstring` functions for any type that provides a stream output operator and for [any basic_string specialisation](#),
- remains consistent and [mostly compatible](#) with these functions,
- provides [extra formatting and concatenation capabilities](#),
- is conceptually simple by building upon the familiar `ostringstream` solution.

Formatting capabilities

One notable feature of the proposed solution is that it is a variadic function template. This doesn't make the interface or the

implementation much more complicated, yet it provides some benefits in terms of flexibility and efficiency. First, this allows for providing of additional formatting instructions using the familiar stream manipulators:

```
int byte = 255;
to_string(hex, uppercase, byte); // format as an uppercase hexadecimal number: "FF"
```

Second, it allows for concise and possibly more efficient concatenation of several objects' representations into a single string:

```
int major = 4, minor = 11, revision = 2;
to_string(major, '.', minor, '.', revision); // yields "4.11.2"
```

An example combining the two use cases shows how this simple solution is at the same time quite powerful:

```
int hours = 4, minutes = 9; bool afternoon = true;
to_string(hours, ':', setfill('0'), setw(2), minutes, afternoon ? " PM" : " AM"); // "4:09 PM"
```

There is yet another use case of the proposed solution: it allows to temporarily reset all formatting flags to the defaults when using an output stream. This may be useful to ensure certain formatting for selected output operations without having to set and then reset all possibly relevant formatting flags. It also allows to treat a subset of output operations as an atomic part, managed by some formatting flags as a whole. Consider:

```
cout << right << setfill('_') << setw(8) <<
to_string(hours, ':', setfill('0'), setw(2), minutes); // "____4:09"
```

The `right`, `setfill('_')` and `setw(8)` directives don't affect formatting of elements of "4:09" substring, but rather of the substring as a whole. Also, the `setfill('0')` and `setw(2)` directives have no effect outside of the substring generation.

Support for any `basic_string` specialisation

Proposed `to_string` and `to_wstring` function templates delegate the real work to a more generic `to_basic_string` function template. Calling the latter directly allows for customisation of the character type, traits and allocator used by the employed stream and the resulting string beyond the two default configurations provided by `to_string` and `to_wstring`.

IV. Impact on the Standard

In principle, proposed function templates' specialisations for built-in numeric types yield results consistent with the already existing `to_string/to_wstring` overloads. The old and new functions could coexist, relying on the overload resolution to prefer a non-templated (existing) version in case of a matching argument type. However, a compatibility problem may arise in case of some distinct but implicitly convertible argument types:

```
to_string(0); // before: calls to_string(int), now: calls to_string(int)
to_string(false); // before: calls to_string(int), now: calls to_string<bool>(bool&&)
to_string('0'); // before: calls to_string(int), now: calls to_string<char>(char&&)
```

While the effect is identical in the first two cases (the result is always "0"), in the last one the result will change from "48" (assuming ASCII encoding) to "0". There are several ways to deal with problematic specialisation cases like this one:

- rename the proposed function templates, e.g. to `to_str/to_wstr`, `format/wformat` OR `stringize/wstringize`,
- add the remaining overloads of `to_string/to_wstring` functions supporting the existing behaviour, e.g.

```
string to_string(char c) { return to_string(static_cast<int>(c)); }
```

- the templated version call can still be forced with the syntax: `to_string<>('0')`,
- explicitly specify the behaviour of new functions as always falling back to the existing ones in case of implicitly convertible argument types,
- ignore the problem for the sake of consistency with I/O Streams behaviour (possibly even deprecating the existing functions in favour of the templated ones).

The paper currently reflects the last option (excluding deprecation), but the author is aware of the significance of backwards compatibility and seeks guidance from the Committee members regarding their preferred solution.

V. Design Decisions

Empty argument list case

There is a question whether calls with empty argument list (i.e. `to_string()`), yielding an empty string, should be explicitly

disallowed. The author's opinion is no – this specialisation, even if lacking obvious use cases, is consistent with the general concept and there are no obvious disadvantages to keeping it. Also, providing such a somewhat artificial limitation could be potentially disruptive in generic code.

Error handling

The functions set the exception flags (which are not set by default) on the stream object in order to detect and signal any potential errors encountered during the formatting operation rather than silently ignore them.

Performance

Known issue of the `ostreamstream` solution used under the hood by the proposed function templates is its relatively poor performance. However, in accordance with the “as if” rule, implementations are free to provide (and probably would benefit a lot from) optimisations for specific argument types, e.g. using `sprintf` directly just like the existing `to_string`/`to_wstring` functions or to preallocating the string's storage if maximum length of all the arguments can be determined upfront.

VI. Technical Specifications

Proposed wording will be provided in a future revision of the document.